

QUAID-I-AZAM UNIVERSITY ISLAMABAD PROJECT



**SUBMITTED BY
AWAIS ALI: 04072212045**

**SUBMITTED TO:
DR. IMRAN**

Course: Cloud DevOps

**DEPARTMENT OF COMPUTER SCIENCE
QUAID-I-AZAM UNIVERSITY, ISLAMABAD**

Task 1 Report: Dockerizing Project

Objective

The objective of this task was to dockerize the project so that all components such as frontend, backend, database, and load balancer run together using Docker and Docker Compose.

Project Description

For this task, a multi-container application **nginx-nodejs-redis** was used.

1. **Node.js**: Backend & frontend
2. **Redis**: Database
3. **Nginx**: Load balancer
4. **Docker Compose**: To manage all containers

Procedure

Step 1: Install and Verify Docker

Docker was installed on the system. To verify installation, the following command was used:

```
docker --version
```

This confirmed that Docker was successfully installed.

Step 2: Open Project Directory

The terminal was opened and the project folder containing the Docker files was accessed:

```
cd nginx-nodejs-redis
```

This folder contained the docker-compose.yml file.

Step 3: Build and Run Containers

All application services were built and started using Docker Compose:

```
docker compose up -d
```

This command created and started containers for Nginx, Node.js applications, and Redis.

Step 4: Check Running Containers

To verify that all containers were running, the following command was used:

```
docker compose ps
```

This showed all active containers and their status.

Step 5: Check Application Logs

Logs were checked to ensure the application was running without errors:

```
docker compose logs
```

Step 6: Verify Redis Database

The Redis container was tested to confirm database connectivity:

```
docker compose exec redis redis-cli ping
```

The output PONG confirmed Redis was working correctly.

Step 7: Verify Application in Browser

The application was tested by opening the browser and navigating to:

```
http://localhost:8080
```

The web application loaded successfully, and the visit counter increased on refresh.

Step 8: Stop Containers

After testing, all containers were stopped using:

```
docker compose down
```

Result

The final year project was successfully dockerized. All components ran inside containers and worked together using Docker Compose.

```
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th se
mester/devops/ABB/devOps/Project/nginx-nodejs-redis$ docker compose up -d
[+] Running 4/4
  ✓ Container redis-db    Running
    0.0s
  ✓ Container web1       Running
  ✓ Container web2       Running
    0.0s
  ✓ Container nginx-lb   Running
    0.0s
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th se
mester/devops/ABB/devOps/Project/nginx-nodejs-redis$ docker compose up -d
[+] Running 4/4
  ✓ Container redis-db    Running
    0.0s
  ✓ Container web2       Healthy
    0.6s
  ✓ Container web1       Healthy
    0.6s
  ✓ Container nginx-lb   Running
    0.0s
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th se
mester/devops/ABB/devOps/Project/nginx-nodejs-redis$
```

```
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/projec
t/nginx-nodejs-redis$ docker compose up -d
[+] Running 5/5
  ✓ Network nginx-nodejs-redis_app-network    Created
    0.1s
  ✓ Container redis-db    Started
    1.0s
  ✓ Container web2       Healthy
    6.8s
  ✓ Container web1       Healthy
    6.8s
  ✓ Container nginx-lb   Started
    6.8s
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/projec
t/nginx-nodejs-redis$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
nginx-lb            nginx-nodejs-redis-nginx "/docker-entrypoint..." nginx       11 seconds ago Up 4 seconds  0.0.0
.0:8080->80/tcp, [::]:8080->80/tcp
redis-db            redis:7-alpine       "docker-entrypoint.s..." redis      11 seconds ago Up 10 seconds  0.0.0
.0:6379->6379/tcp, [::]:6379->6379/tcp
web1                nginx-nodejs-redis-web1 "docker-entrypoint.s..." web1      11 seconds ago Up 9 seconds (healthy) 0.0.0
.0:81->5000/tcp, [::]:81->5000/tcp
web2                nginx-nodejs-redis-web2 "docker-entrypoint.s..." web2      11 seconds ago Up 9 seconds (healthy) 0.0.0
.0:82->5000/tcp, [::]:82->5000/tcp
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/projec
t/nginx-nodejs-redis$
```

localhost:8080

Gmail YouTube Maps All Bookmarks

Dockerized Application

Node.js + Redis + Nginx Load Balancer

Visit Counter

6

TOTAL VISITS

Server: web2

Last Updated: 24/12/2025, 13:24:28

1:24 PM

Task 2 Report: Kubernetes Monitoring and Ingress Setup

Objective

The objective of this task was to deploy the application on Kubernetes and configure **Ingress, Monitoring, Metrics Server, and Grafana** using **Helm charts**, and then display Kubernetes metrics using Grafana dashboards.

Procedure

Step 1: Build Docker Images

Docker images for the Node.js application and Nginx were created.

```
docker build -t awais3823/nginx-nodejs-redis-web:latest ./web
```

```
docker build -t awais3823/nginx-nodejs-redis-nginx:latest ./nginx
```

Step 2: Push Images to Docker Hub

Images were uploaded to Docker Hub so Kubernetes could pull them.

```
docker login
```

```
docker push awais3823/nginx-nodejs-redis-web:latest
```

```
docker push awais3823/nginx-nodejs-redis-nginx:latest
```

Step 3: Create Kubernetes Namespace

A separate namespace was created for the application.

```
kubectl apply -f k8s/namespace.yaml
```

Step 4: Deploy Application Components

Redis, Node.js web application, and Nginx were deployed.

```
kubectl apply -f k8s/redis-deployment.yaml
```

```
kubectl apply -f k8s/web-deployment.yaml
```

```
kubectl apply -f k8s/nginx-deployment.yaml
```

Step 5: Verify Pods and Services

Checked if pods and services were running correctly.

```
kubectl get pods -n nginx-nodejs-redis
```

```
kubectl get svc -n nginx-nodejs-redis
```

Step 6: Configure Nginx Ingress Controller (Helm)

Nginx Ingress Controller was installed using Helm.

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
```

```
helm repo update
```

```
helm install ingress-nginx ingress-nginx/ingress-nginx \
```

```
--namespace ingress-nginx \
```

```
--create-namespace
```

Step 7: Apply Ingress Resource

Ingress rules were applied to expose the application.

```
kubectl apply -f k8s/ingress.yaml
```

Host entry was added for local access.

```
echo "<NODE_IP> nginx-nodejs-redis.local" | sudo tee -a /etc/hosts
```

Step 8: Access Application

The application was accessed using Ingress.

```
http://nginx-nodejs-redis.local
```

```
http://nginx-nodejs-redis.local/api/visits
```

Step 9: Install Monitoring Stack (Prometheus + Grafana)

Monitoring was configured using Helm chart.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

```
helm install monitoring prometheus-community/kube-prometheus-stack \
```

```
-n monitoring --create-namespace
```

Step 10: Verify Monitoring Pods

Checked monitoring components.

```
kubectl get pods -n monitoring
```

Step 11: Access Grafana Dashboard

Grafana was accessed using NodePort.

```
kubectl get nodes -o wide
```

<http://172.29.254.201:30000>

Username: admin

Password: admin

Step 12: Import Kubernetes Dashboard

A Kubernetes dashboard from the **Grafana official repository** was imported and connected with **Prometheus** to display cluster metrics such as:

- CPU usage
- Memory usage
- Pod status
- Node health

Verification

```
kubectl get ingress -n nginx-nodejs-redis
```

```
kubectl get nodes -o wide
```

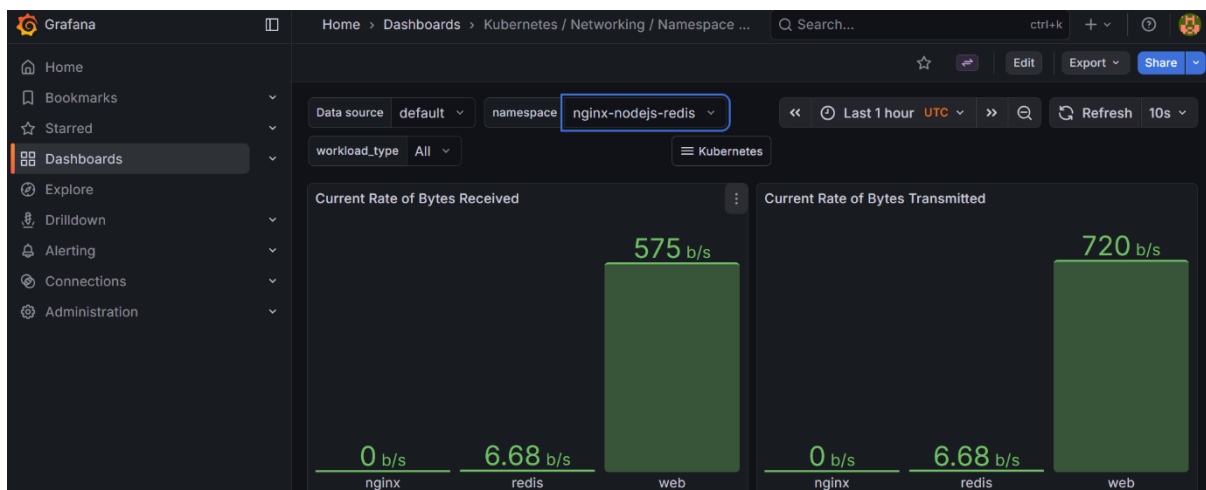
Application accessed at:

<http://nginx-nodejs-redis.local>

http://<NODE_IP>:30000

```
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/Project/nginx-nodejs
s-redis$ docker search awais3823/nginx-nodejs-redis-web
NAME                                DESCRIPTION                                STARS    OFFICIAL
nginx/nginx-ingress                 NGINX and NGINX Plus Ingress Controllers fo... 111
redis/redis-stack                   redis-stack installs a Redis server with add... 158
redis/redis-stack-server            redis-stack-server installs a Redis server w... 99
nginx/nginx-prometheus-exporter     NGINX Prometheus Exporter for NGINX and NGIN... 50
redislabs/redis                     Clustered in-memory database engine compatib... 44
nginxinc/nginx-unprivileged         Unprivileged NGINX Dockerfiles              176
redis/redisinsight                 Redis Insight - our best official GUI for Re... 45
redislabs/redisearch               Redis With the RedisSearch module pre-loaded... 68
redislabs/redis-py                  5
nginx/nginx-ingress-operator        NGINX Ingress Operator for NGINX and NGINX P... 3
nginxinc/nginx-s3-gateway           Authenticating and caching gateway based on ... 6
nginx/unit                          This repository is retired, use the Docker o... 65
redislabs/redisinsight              RedisInsight - The GUI for Redis             116
redislabs/redismod                  An automated build of redismod - latest Redi... 51
redislabs/ng-redis-raft             Redis with redis raft module                 0
redis                               Redis is the world's fastest data platform f... 13491 [OK]
redislabs/redis-webcli              A tiny Flask app to provide access to Redis ... 5
redislabs/redisgraph                A graph database module for Redis            28
redislabs/redisai                   6
nginx/nginx-quic-qns                NGINX QUIC interop                          1
redislabs/redis-di-cli              2
redislabs/redisgears                An automated build of RedisGears             4
redislabs/redistimeseries            A time series database module for Redis      14
redislabs/pcf-redis-enterprise-tester 0
redislabs/redis-connect-cli         0
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/Project/nginx-nodejs
s-redis$ docker search awais3823/nginx-nodejs-redis-web
```

```
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/Project/nginx-nodejs
s-redis$
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/Project/nginx-nodejs
s-redis$ kubectl get pods -n monitoring
NAME                                READY    STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0 2/2      Running   0           22m
monitoring-grafana-56c6d498b8-f295c 3/3      Running   0           20m
monitoring-kube-prometheus-operator-67fdb54dd6-9c9tt 1/1      Running   0           22m
monitoring-kube-state-metrics-7f575674-zptjz 1/1      Running   11 (41m ago) 12d
prometheus-monitoring-kube-prometheus-prometheus-0 2/2      Running   0           20m
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/Project/nginx-nodejs
s-redis$ kubectl get svc -n monitoring
NAME                                TYPE           CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
alertmanager-operated               ClusterIP      None           <none>          9093/TCP,9094/TCP,9094/UDP             12d
monitoring-grafana                  NodePort       10.105.134.253 <none>          80:30000/TCP                           12d
monitoring-kube-prometheus-alertmanager NodePort       10.105.208.243 <none>          9093:30093/TCP,8080:31346/TCP          12d
monitoring-kube-prometheus-operator ClusterIP       10.96.39.229   <none>          443/TCP                                12d
monitoring-kube-prometheus-prometheus NodePort       10.99.70.177   <none>          9090:30090/TCP,8080:30413/TCP          12d
monitoring-kube-state-metrics        ClusterIP      10.104.105.141 <none>          8080/TCP                                12d
prometheus-operated                 ClusterIP      None           <none>          9090/TCP                                12d
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/ABB/devOps/Project/nginx-nodejs
s-redis$ |
```





Task 3 Report: GitHub CI/CD Pipeline

Objective

The objective of this task was to create a **CI/CD pipeline using GitHub Actions** that automatically **builds a new Docker image** and **pushes it to Docker Hub** whenever a **new GitHub release** is created.

The pipeline also **deploys the new image to Kubernetes automatically**.

Procedure

Step 1: Create GitHub Workflow

A GitHub Actions workflow file was created in the repository at:

`.github/workflows/ci-cd.yaml`

This workflow runs automatically when a **new release is published** on GitHub.

Step 2: Configure GitHub Secrets

Secrets were added in GitHub to securely store credentials.

1. DOCKERHUB_USERNAME
2. DOCKERHUB_PASSWORD (Docker Hub access token)
3. KUBECONFIG (base64 encoded Kubernetes config)
`cat ~/.kube/config | base64`

These secrets allow GitHub Actions to:

1. Login to Docker Hub

2. Push images
3. Deploy to Kubernetes

Step 3: Trigger Pipeline on Release

The pipeline is triggered when a **new GitHub release** is created.

Example release tags:

v1.0.0

v1.1.0

Step 4: Build Docker Images

The workflow automatically builds Docker images for:

- Node.js web application
- Nginx server

Images are tagged as:

- latest
- release version (e.g. 1.0.0)

Step 5: Push Images to Docker Hub

After building, the images are pushed to Docker Hub.

Example images:

awais382/nginx-nodejs-redis-web:latest

awais382/nginx-nodejs-redis-web:1.0.0

awais382/nginx-nodejs-redis-nginx:latest

awais382/nginx-nodejs-redis-nginx:1.0.0

Step 6: Deploy Updated Images to Kubernetes

The pipeline updates Kubernetes deployments automatically using kubectl.

- Web deployment updated with new image
- Nginx deployment updated with new image

Kubernetes then pulls the **new Docker images** and redeploys the application.

Step 7: Verify Deployment

The workflow checks:

- Deployment rollout status
- Pod health
- Running image versions

This ensures the deployment is successful.

Verification

```
kubectl get deployments -n nginx-nodejs-redis
```

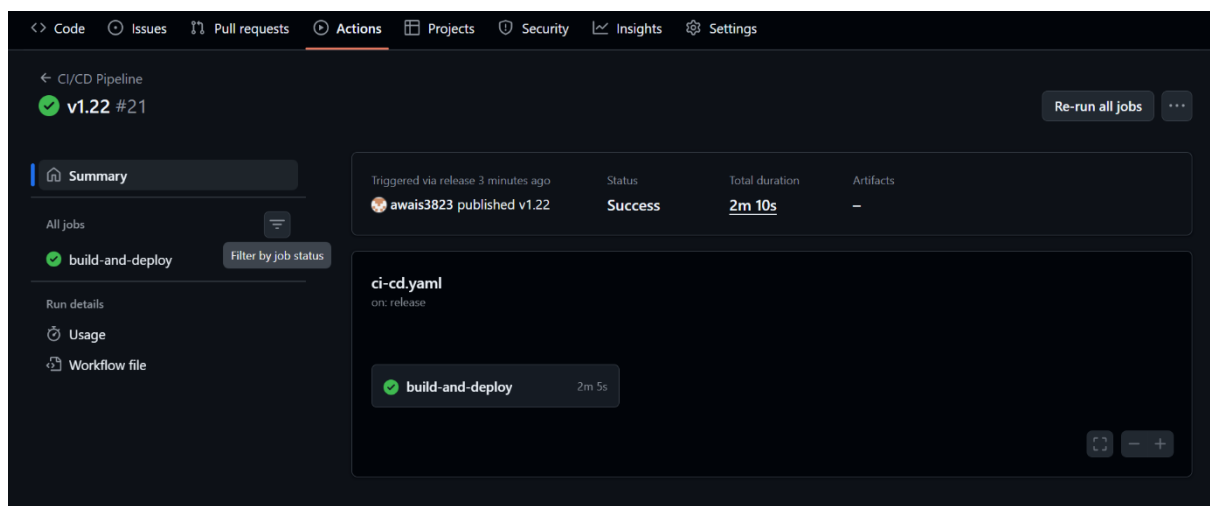
```
kubectl get pods -n nginx-nodejs-redis
```

Application is accessed through:

```
http://nginx-nodejs-redis.local
```

Result

1. CI/CD pipeline successfully implemented
2. Docker images are built automatically
3. Images are pushed to Docker Hub
4. Kubernetes is updated automatically
5. Manual deployment is no longer required



Summary

All jobs

build-and-deploy

Run details

Usage

Workflow file

build-and-deploy

succeeded 1 minute ago in 2m 5s

Search logs

> Set up job15s

> Checkout repository2s

> Log in to Docker Hub8s

> Build and Push Web Image19s

> Build and Push Nginx Image17s

> Configure Kubernetes1s

> Deploy to Kubernetes43s

> Post Log in to Docker Hub8s

> Post Checkout repository0s

> Complete job0s